Theory and Grid Design for a Non-Orthogonal C-Grid Vector-Invariant Scheme for Oceananigans.jl and Breeze.jl

Derivation, implementation design note, and staggered-Hodge test plan

May 16, 2026

Abstract

This note develops the mathematical and code-design foundations for a non-orthogonal C-grid momentum advection scheme for Oceananigans.jl and Breeze.jl. The target is a thin-atmosphere spherical-shell grid, beginning with one panel of the equiangular gnomonic cubed sphere. Two dynamical regimes are in scope: a fully compressible implementation in which the conservative mass-flux divergence is generally nonzero, and a hydrostatic-incompressible implementation in which the compatible volume-flux divergence vanishes in fixed physical coordinates. The central distinction is between model velocities, transport velocities, and transport fluxes. Momentum dynamics and vorticity use covariant velocity components, while scalar advection and continuity use metric-compatible contravariant or area-normal transport velocities and fluxes. This distinction is already present in Oceananigans' HydrostaticFreeSurfaceModel, where tracer tendencies use `model.transport_velocities`, while momentum tendencies use `model.velocities`. Breeze should adopt the same abstraction through `transport_velocities(model)` and `transport_fluxes(model)`. The algorithmic abstraction is a generic `NonOrthogonalVectorInvariant`, with `NonOrthogonalWENOVectorInvariant` as a WENO-specialized constructor. The materialized non-orthogonal scheme owns the geometric state fields required by the operator: covariant velocity, contravariant velocity, transport velocities, volume transports, mass transports, transport divergences, kinetic energy, and vorticity two-forms. This is a deliberate extension of the current stateless Oceananigans `WENOVectorInvariant` design.

Contents

1 Scope and guiding principles

The goal is to extend the structured C-grid finite-volume machinery in Oceananigans and Breeze to non-orthogonal logically rectangular grids with minimal disruption, while avoiding false equivalences that are harmless on orthogonal grids but wrong on non-orthogonal grids. The key design rule is

$$\boxed{\text{model velocity variables are not necessarily scalar transport velocities.}} \tag{1}$$

On an orthogonal grid, these objects often coincide. On a non-orthogonal grid they do not.

The model interface should distinguish:

`model.velocities` model velocity variables used by momentum, closures, forcing, diagnostics, and output.

`transport_velocities(model)` velocity tuple used for scalar, tracer, moisture, and thermodynamic advection.

`transport_fluxes(model)` volume or mass fluxes used by continuity and vector-invariant coupling.

For simple orthogonal cases, `transport_velocities(model) = model.velocities` may be an alias. For non-orthogonal grids, `transport_velocities(model)` is diagnosed from the model velocities through a metric or Hodge map.

This convention follows an existing Oceananigans pattern. In HydrostaticFreeSurfaceModel (HFSM), tracer tendencies are explicitly routed through `model.transport_velocities`, while momentum tendencies are computed using `model.velocities` [8]. Breeze already has the function-interface version in embryo: `transport_velocities(model) = model.velocities` and `advecting_momentum(model) = model.momentum`, with terrain-following compressible dynamics overriding the vertical transport velocity and momentum [1, 2]. The non-orthogonal effort should promote this split into a first-class design rule.

2 Continuous non-orthogonal geometry

Let ξ^1, ξ^2, ξ^3 be logical coordinates and let

$$\boldsymbol{x} = \boldsymbol{x}(\xi^1, \xi^2, \xi^3) \tag{2}$$

be a time-independent coordinate map. Define covariant and contravariant basis vectors

$$\boldsymbol{a}_i = \partial_i \boldsymbol{x}, \qquad \boldsymbol{a}^i = \nabla \xi^i, \qquad \boldsymbol{a}^i \cdot \boldsymbol{a}_j = \delta^i_j. \tag{3}$$

The metric tensors and Jacobian are

$$g_{ij} = \boldsymbol{a}_i \cdot \boldsymbol{a}_j, \qquad g^{ij} = \boldsymbol{a}^i \cdot \boldsymbol{a}^j, \qquad J = \boldsymbol{a}_1 \cdot (\boldsymbol{a}_2 \times \boldsymbol{a}_3). \tag{4}$$

Velocity components are

$$u_i = \boldsymbol{u} \cdot \boldsymbol{a}_i, \qquad u^i = \boldsymbol{u} \cdot \boldsymbol{a}^i, \qquad u^i = g^{ij} u_j. \tag{5}$$

The covariant components u_i are the coefficients of the velocity one-form. They are the natural variables for circulation and vorticity. The contravariant components u^i are the natural variables for finite-volume transport because

$$\nabla \cdot \boldsymbol{u} = \frac{1}{J} \partial_i (J u^i). \tag{6}$$

Two integrated transport objects will be used throughout this note:

$$\mathcal{V}^i = J u^i, \qquad \text{volume transport,} \tag{7}$$

$$\mathcal{U}^i = J \rho u^i = \rho \mathcal{V}^i, \qquad \text{mass transport.} \tag{8}$$

The notation $\mathcal{U}^i$ is chosen because the geophysical fluid dynamics literature often uses U, V, W for transported fluxes; the calligraphic font distinguishes the transport from the pointwise velocity u^i. The earlier symbol M^i is avoided below because it can obscure whether the flux is a mass flux, volume flux, or an area-weighted velocity.

In a finite-volume code that multiplies velocities by face areas, it is also useful to define the area-normal transport velocity $u^i_\perp$ by

$$A_i u^i_\perp = \mathcal{V}^i = J u^i, \qquad \mathcal{U}^i = \rho A_i u^i_\perp. \tag{9}$$

The choice between storing raw contravariant velocity u^i, area-normal velocity $u^i_\perp$, volume transport $\mathcal{V}^i$, or mass transport $\mathcal{U}^i$ is an implementation choice. The invariant requirement is that scalar advection and continuity use a single conservative transport object.

3 Thin-atmosphere equiangular gnomonic cubed-sphere panel

3.1 Panel mapping

The first non-orthogonal grid target should be one panel of an equiangular gnomonic cubed sphere. This is the geometry used by ClimaCore's `Meshes.EquiangularCubedSphere`, whose mapping uses `tanpi(phi_x / 4)` and `tanpi(phi_y / 4)` before rotating to one of six panels [5]. ClimaAtmos builds its spherical grid by creating such a mesh, constructing a two-dimensional topology, and then using `CommonGrids.ExtrudedCubedSphereGrid` [4, 6].

For one north-pole panel, use equiangular coordinates

$$\alpha, \beta \in \left[-\frac{\pi}{4}, \frac{\pi}{4}\right], \qquad p = \tan\alpha, \qquad q = \tan\beta, \qquad D = 1 + p^2 + q^2. \tag{10}$$

The unit-sphere map is

$$\widehat{\boldsymbol{x}}(\alpha, \beta) = \frac{1}{\sqrt{D}} \begin{pmatrix} p \\ q \\ 1 \end{pmatrix}. \tag{11}$$

A spherical shell coordinate is

$$\boldsymbol{x}(\alpha, \beta, r) = r\widehat{\boldsymbol{x}}(\alpha, \beta). \tag{12}$$

For a thin-atmosphere approximation, the horizontal metric can be evaluated at a fixed radius R rather than $R + z$. The deep-shell form keeps $r = R + z$ in horizontal metric factors.

3.2 Analytic horizontal metric

For the shell map (12), the horizontal covariant metric components are

$$g_{\alpha\alpha} = r^2 \frac{(1+p^2)^2(1+q^2)}{D^2}, \tag{13}$$

$$g_{\beta\beta} = r^2 \frac{(1+q^2)^2(1+p^2)}{D^2}, \tag{14}$$

$$g_{\alpha\beta} = -r^2 \frac{pq(1+p^2)(1+q^2)}{D^2}. \tag{15}$$

The off-diagonal term $g_{\alpha\beta}$ is generically nonzero. This is the simplest structured spherical grid on which a non-orthogonal C-grid algorithm must confront the difference between covariant velocity and transport velocity.

The horizontal metric determinant is

$$g_h = g_{\alpha\alpha} g_{\beta\beta} - g_{\alpha\beta}^2 = r^4 \frac{(1+p^2)^2(1+q^2)^2}{D^3}, \tag{16}$$

so

$$\sqrt{g_h} = r^2 \frac{(1+p^2)(1+q^2)}{D^{3/2}}. \tag{17}$$

The inverse horizontal metric is

$$g^{\alpha\alpha} = \frac{D}{r^2(1+p^2)}, \tag{18}$$

$$g^{\beta\beta} = \frac{D}{r^2(1+q^2)}, \tag{19}$$

$$g^{\alpha\beta} = \frac{pqD}{r^2(1+p^2)(1+q^2)}. \tag{20}$$

For the simplest spherical-shell panel,

$$g_{\alpha r} = g_{\beta r} = 0, \qquad g_{rr} = 1. \tag{21}$$

Terrain-following or more general vertical coordinates can be treated later by adding $g_{\alpha r}$ and $g_{\beta r}$ terms.

3.3 Panel rotations and six-panel geometry

A single-panel prototype can work entirely with (11). A six-panel grid requires a rotation map

$$\boldsymbol{x}_P = \mathcal{R}_P \boldsymbol{x}_1, \tag{22}$$

where $P = 1, \ldots, 6$ is the panel number. ClimaCore implements this through `to_panel(panel, coord)` and `from_panel(panel, coord)` for cubed-sphere meshes [5]. Oceananigans' current conformal cubed-sphere panel code similarly supports panel rotations and constructs staggered longitude and latitude arrays from a CubedSphere mapping [14]. The non-orthogonal finite-volume implementation should reuse the idea of panel rotations but not assume orthogonal metric factors.

4 C-grid locations and discrete geometry

Following Oceananigans operator notation, staggered locations are denoted by lower-case superscripts rather than tuple notation. The symbol c denotes a cell center and f denotes a face. Thus a cell-centered scalar lives at $^{\mathrm{ccc}}$, the first velocity component lives at $^{\mathrm{fcc}}$, the second component at $^{\mathrm{cfc}}$, and the third component at $^{\mathrm{ccf}}$. This mirrors Oceananigans source notation, where operator names carry Unicode location superscripts; in LaTeX we write these as, for example, $\mathrm{div}^{\mathrm{ccc}}$, Δx^{fcc}, $(V^{-1})^{\mathrm{fcc}}$, and ζ_3^{ffc}. In this note,

$$u_1^{\mathrm{fcc}}, \qquad u_2^{\mathrm{cfc}}, \qquad u_3^{\mathrm{ccf}} \tag{23}$$

denote component-staggered covariant velocities. For tensor components, a comma separates tensor indices from the location. For example, $g^{12,\mathrm{fcc}}$ means the $(1,2)$ component of the contravariant metric evaluated at $^{\mathrm{fcc}}$.

The basic C-grid placement is:

Quantity	Mathematical role	Oceananigans location
$\rho, \chi, K, \mathcal{D}_{\mathcal{V}}, \mathcal{D}_{\mathcal{U}}$	cell-centered scalars	ccc
$u_1, \mathcal{V}^1, \mathcal{U}^1$	first component / first-face transport	fcc
$u_2, \mathcal{V}^2, \mathcal{U}^2$	second component / second-face transport	cfc
$u_3, \mathcal{V}^3, \mathcal{U}^3$	third component / third-face transport	ccf
ζ_{12}, Z_{12}	horizontal vorticity two-form	ffc
ζ_{13}, Z_{13}	vertical shear two-form	fcf
ζ_{23}, Z_{23}	vertical shear two-form	cff

A non-orthogonal grid requires at least the following metric information:

$$J \quad \text{at cell centers and transport faces,} \tag{24}$$

$$g_{ij}, g^{ij} \quad \text{at locations required by Hodge maps,} \tag{25}$$

$$A_i \quad \text{face areas, if velocity-based flux operators are reused,} \tag{26}$$

$$\text{edge lengths and dual lengths, if a TRiSK-like Hodge is pursued.} \tag{27}$$

The first implementation should use a tensor-metric Hodge map. A TRiSK-like discrete exterior calculus formulation can be developed later if it becomes advantageous for panel connectivity or conservation properties. The tensor-metric path is simpler for Oceananigans' structured finite-volume operators and easier to test against analytic equiangular metrics.

5 Velocity, transport velocity, and transport diagnostics

The model momentum variables should represent covariant momentum or velocity-like quantities compatible with covariant dynamics. From these, the non-orthogonal advection object diagnoses the transport quantities. In continuous notation,

$$u^i = g^{ij} u_j, \qquad \mathcal{V}^i = J u^i, \qquad \mathcal{U}^i = \rho \mathcal{V}^i = J \rho u^i. \tag{28}$$

Here $\mathcal{V}^i$ is a volume transport in computational coordinates and $\mathcal{U}^i$ is a mass transport. If velocity-based scalar kernels are reused, then

$$u^i_\perp = \frac{\mathcal{V}^i}{A_i} = \frac{J}{A_i} u^i. \tag{29}$$

For a horizontally non-orthogonal, vertically orthogonal spherical shell, the continuous metric conversion reduces to

$$u^1 = g^{11} u_1 + g^{12} u_2, \tag{30}$$
$$u^2 = g^{21} u_1 + g^{22} u_2, \tag{31}$$
$$u^3 = g^{33} u_3. \tag{32}$$

Discretely, this should not be hard-coded as a single multiply/interpolate convention. It should be represented as Hodge blocks between Oceananigans locations, for example

$$\mathcal{V}^{1,\mathrm{fcc}} = \mathcal{H}^{11}_{\mathrm{fcc}\leftarrow\mathrm{fcc}} u_1^{\mathrm{fcc}} + \mathcal{H}^{12}_{\mathrm{fcc}\leftarrow\mathrm{cfc}} u_2^{\mathrm{cfc}}. \tag{33}$$

A target-metric discretization would take $\mathcal{H}^{12} u_2$ to be $G^{12,\mathrm{fcc}} \mathcal{I}_{\mathrm{cfc}\to\mathrm{fcc}} u_2$, while a product-interpolated discretization would take it to be $\mathcal{I}_{\mathrm{cfc}\to\mathrm{fcc}}(G^{12,\mathrm{cfc}} u_2^{\mathrm{cfc}})$. Section 6 lays out the Hodge-map candidates and the tests used to select among them.

Transport divergences are written explicitly as

$$\mathcal{D}_\mathcal{V} \equiv \mathcal{D}_i \mathcal{V}^i = \mathcal{D}_1 \mathcal{V}^1 + \mathcal{D}_2 \mathcal{V}^2 + \mathcal{D}_3 \mathcal{V}^3, \tag{34}$$
$$\mathcal{D}_\mathcal{U} \equiv \mathcal{D}_i \mathcal{U}^i = \mathcal{D}_1 \mathcal{U}^1 + \mathcal{D}_2 \mathcal{U}^2 + \mathcal{D}_3 \mathcal{U}^3. \tag{35}$$

Horizontal divergences are denoted

$$\mathcal{D}_{h,\mathcal{V}} = \mathcal{D}_1 \mathcal{V}^1 + \mathcal{D}_2 \mathcal{V}^2, \tag{36}$$
$$\mathcal{D}_{h,\mathcal{U}} = \mathcal{D}_1 \mathcal{U}^1 + \mathcal{D}_2 \mathcal{U}^2. \tag{37}$$

This notation deliberately avoids a bare symbol like C. The relevant divergence is always either a volume-transport divergence $\mathcal{D}_\mathcal{V}$ or a mass-transport divergence $\mathcal{D}_\mathcal{U}$, and the horizontal split is $\mathcal{D}_{h,\mathcal{V}}$ or $\mathcal{D}_{h,\mathcal{U}}$.

6 Staggered metric placement, Hodge maps, and empirical verification

The most delicate part of the non-orthogonal C-grid implementation is not the continuous identity

$$u^i = g^{ij} u_j. \tag{38}$$

The delicate part is the staggered discrete Hodge map that converts component-staggered covariant velocity fields into transport velocities and transports at the correct Oceananigans locations. This map must be treated as a primary numerical operator. We should not decide between candidate stencils by taste.

The robust design principle is

> Define a discrete Hodge map, then verify it by consistency, adjointness, positivity, and free-stream tests.

6.1 Oceananigans C-grid locations

We use Oceananigans-style location superscripts. A superscript such as $^{\mathrm{fcc}}$ means face in the first coordinate, center in the second, and center in the third. Thus

$$u_1^{\mathrm{fcc}}, \qquad u_2^{\mathrm{cfc}}, \qquad u_3^{\mathrm{ccf}} \tag{39}$$

are the staggered covariant velocity components. The transport components should live at the same face locations:

$$\mathcal{V}^{1,\mathrm{fcc}}, \qquad \mathcal{V}^{2,\mathrm{cfc}}, \qquad \mathcal{V}^{3,\mathrm{ccf}}. \tag{40}$$

The vorticity two-forms live at edge locations:

$$\zeta_{12}^{\mathrm{ffc}}, \qquad \zeta_{13}^{\mathrm{fcf}}, \qquad \zeta_{23}^{\mathrm{cff}}. \tag{41}$$

The same locations are used for the planetary vorticity two-forms f_{ij} and absolute-vorticity/PV-numerator two-forms

$$Z_{ij} = \zeta_{ij} + f_{ij}. \tag{42}$$

For a horizontally non-orthogonal but vertically orthogonal thin shell, the minimal metric layout is

Location	Metric/geometry fields	Main use
ccc	$J, g_{ij}, g^{ij}, \Delta V$	scalar volumes, kinetic energy, diagnostics
fcc	$J, A_1, G^{11}, G^{12}, G^{13}$	$\mathcal{V}^1, \mathcal{U}^1, u^1, u^1_\perp$
cfc	$J, A_2, G^{21}, G^{22}, G^{23}$	$\mathcal{V}^2, \mathcal{U}^2, u^2, u^2_\perp$
ccf	$J, A_3, G^{31}, G^{32}, G^{33}$	$\mathcal{V}^3, \mathcal{U}^3, u^3, u^3_\perp$
ffc	coordinate spacings, f_{12}	ζ_{12}, Z_{12}
fcf	coordinate spacings, f_{13}	ζ_{13}, Z_{13}
cff	coordinate spacings, f_{23}	ζ_{23}, Z_{23}

Here

$$G^{ij} \equiv J g^{ij} \tag{43}$$

is the contravariant metric density. It is often the more natural coefficient for volume transport because

$$\mathcal{V}^i = J u^i = J g^{ij} u_j = G^{ij} u_j. \tag{44}$$

For the first thin-shell target, $G^{13} = G^{23} = 0$ and G^{33} is purely vertical. The table is written in full three-dimensional form so that the notation remains valid if deep-atmosphere or terrain-following effects are added later.

6.2 The Hodge map, not a pointwise interpolation convention

The discrete metric conversion should be written as a Hodge map:

$$\begin{pmatrix} \mathcal{V}^1 \\ \mathcal{V}^2 \\ \mathcal{V}^3 \end{pmatrix} = \mathcal{H} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix}. \tag{45}$$

In block form,

$$\mathcal{V}^{i,\ell_i} = \sum_{j=1}^{3} \mathcal{H}^{ij}_{\ell_i \leftarrow \ell_j} u_j^{\ell_j}, \qquad \ell_1 = \text{fcc}, \quad \ell_2 = \text{cfc}, \quad \ell_3 = \text{ccf}. \tag{46}$$

The diagonal blocks are local metric-density multiplications to leading order. The off-diagonal blocks contain both metric information and staggered interpolation. For example, $\mathcal{H}^{12}_{\text{fcc}\leftarrow\text{cfc}}$ maps u_2^{cfc} into the first-direction volume transport $\mathcal{V}^{1,\text{fcc}}$.

There are at least three plausible ways to build this block. The first is a target-metric form:

$$\left(\mathcal{H}^{12}_{\text{target}} u_2\right)^{\text{fcc}} = G^{12,\text{fcc}}\, \mathcal{I}_{\text{cfc}\to\text{fcc}} u_2. \tag{47}$$

The second is a product-interpolated form:

$$\left(\mathcal{H}^{12}_{\text{product}} u_2\right)^{\text{fcc}} = \mathcal{I}_{\text{cfc}\to\text{fcc}} \left(G^{12,\text{cfc}} u_2^{\text{cfc}}\right). \tag{48}$$

The third is an energy-symmetric Hodge map, constructed from a discrete kinetic-energy quadrature rather than chosen as a pointwise formula. Choose quadrature points q, positive weights w_q, metric-density components G_q^{ij}, and interpolation operators $\mathcal{I}_i^q$ from each velocity component location to the quadrature point. Define

$$E_h(\boldsymbol{u}) = \frac{1}{2} \sum_q w_q G_q^{ij} \left(\mathcal{I}_i^q u_i\right) \left(\mathcal{I}_j^q u_j\right). \tag{49}$$

Then define the Hodge map by differentiating this energy with respect to the staggered unknowns:

$$W_i \mathcal{V}^i = \frac{\partial E_h}{\partial u_i}, \tag{50}$$

where W_i is the inner-product weight associated with the u_i location. This construction makes the off-diagonal blocks weighted adjoints by design.

The continuous identity $\mathcal{V}^1 = G^{11}u_1 + G^{12}u_2 + G^{13}u_3$ does not decide between (47) and (48). Both are consistent for smooth fields, but they differ by terms involving metric gradients and velocity gradients. The accepted implementation should be the simplest Hodge map that passes the empirical tests in section 6.6.

6.3 Metric placement: target-location metrics are a candidate, not a theorem

A useful first candidate is to evaluate metric densities at the target transport location. Thus $G^{12,\text{fcc}}$ is used in (47). This is attractive because the resulting transport $\mathcal{V}^{1,\text{fcc}}$ is a face-located object, so all coefficients in that transport live at the same face location. ClimaCore's spectral-element construction follows a related principle: local geometry, Jacobians, weighted Jacobians, and surface geometry are evaluated at the quadrature or surface locations where they are needed [26].

However, target-location metric multiplication is not automatically superior. Product interpolation (48) can be attractive for free-stream preservation, because the metric factor is attached to the source component before interpolation. The correct design question is not

Should we multiply first or interpolate first?

The correct design question is

> Which discrete Hodge map preserves the metric identities, energy inner product, and divergence compatibility required by the dynamics?

This should be made explicit in code by providing candidate Hodge types:

```
abstract type AbstractSphericalShellHodgeMap end

struct TargetMetricHodge <: AbstractSphericalShellHodgeMap end
struct ProductInterpolatedHodge <: AbstractSphericalShellHodgeMap end
struct EnergySymmetricHodge <: AbstractSphericalShellHodgeMap end
```

The implementation should select a default only after the test campaign has ruled out dangerous candidates.

6.4 What TRiSK teaches about staggered metric maps

TRiSK does not answer the multiply-versus-interpolate question by a local pointwise convention. It turns the problem into a constrained operator-design problem. In the shallow-water C-grid setting, mass or layer thickness is stored in primal cells, normal velocity is stored on primal edges, and vorticity/PV is stored on dual cells. The vector-invariant/PV flux term needs a flux component perpendicular to the prognostic normal flux. TRiSK therefore constructs a map from the primal normal edge flux F_e to a perpendicular or dual flux $F_e^{\perp}$:

$$F_e^{\perp} = \frac{1}{d_e} \sum_{e' \in ECP(e)} w_{e,e'}\, l_{e'} F_{e'}. \tag{51}$$

Here $l_{e'}$ and d_e are metric lengths and $w_{e,e'}$ are geometric weights [18]. This is best understood as a Hodge/rotation map between staggered flux spaces, not as a casual interpolation of a pointwise velocity.

The key TRiSK compatibility property is that the dual divergence of the mapped perpendicular flux is an interpolation of the primal divergence of the original flux:

$$D_{\text{dual}} M(F) = I\left(D_{\text{primal}} F\right). \tag{52}$$

This commuting property is what allows a compatible auxiliary thickness equation on the dual mesh, and therefore a PV variable collocated with vorticity. It is also a direct anti-noise property: a divergence-free flux remains divergence-free after the staggered rotation/interpolation, up to the chosen interpolation.

TRiSK also imposes energy neutrality of the nonlinear Coriolis/PV flux. Pairwise antisymmetric edge weights and symmetric PV averages make the vorticity/PV flux do no net work in the centered nondissipative case [18]. In our notation, the corresponding target is a skew-adjoint vorticity operator in the kinetic-energy inner product:

$$\langle u, \mathsf{C}_Z u \rangle_h = 0. \tag{53}$$

There is an important difference. Classical TRiSK assumes a locally orthogonal primal-dual geometry, whereas our equiangular gnomonic `SphericalShellGrid` is a logically rectangular non-orthogonal tensor grid. In our case, the transport Hodge map contains true off-diagonal tensor-metric terms such as $G^{12}u_2$. So we should not copy the TRiSK perpendicular-flux formula directly. The lesson is structural:

Construct the staggered Hodge/rotation map so that divergence, PV, and energy identities commute discretely.

For the structured C-grid, this suggests the following analogues:

$$\mathcal{V}^i = \mathcal{H}^i{}_j u_j, \qquad \text{covariant velocity to volume transport,} \tag{54}$$

$$\mathcal{V}^\perp = \mathsf{R}\mathcal{V}, \qquad \text{transport rotation or dual transport for vorticity fluxes,} \tag{55}$$

$$D_{\text{dual}}\mathsf{R}\mathcal{V} = I(D_{\text{primal}}\mathcal{V}), \qquad \text{commuting divergence target,} \tag{56}$$

$$\langle u, \mathsf{C}_Z u\rangle_h = 0, \qquad \text{energy-neutral centered vorticity flux target.} \tag{57}$$

6.5 Mimetic requirements for the SphericalShellGrid Hodge map

The Hodge-map candidates should be judged against four structural requirements. First, the Hodge map should be consistent with analytic metric tensors:

$$\mathcal{H}u_\flat \approx \mathcal{V}_{\text{exact}} \tag{58}$$

for smooth analytic vector fields. Second, it should preserve constant physical flows. If $\boldsymbol{u}_0$ is a constant Cartesian velocity and $u_i = \boldsymbol{u}_0 \cdot \boldsymbol{a}_i$, then

$$\mathcal{D}_i\left(\mathcal{H}^i{}_j u_j\right) \approx 0. \tag{59}$$

Third, the Hodge map should define a positive kinetic-energy quadratic form:

$$E_h = \frac{1}{2} u^T W \mathcal{H} u > 0 \quad \text{for nonzero velocity.} \tag{60}$$

Fourth, the off-diagonal blocks should be weighted adjoints:

$$W_1 \mathcal{H}^{12} \approx (W_2 \mathcal{H}^{21})^T. \tag{61}$$

This condition is the finite-volume C-grid analogue of an assembled symmetric mass matrix in mimetic finite elements. It is the strongest reason not to accept either (47) or (48) solely because it looks more natural.

The relative vorticity two-forms should remain topological exterior derivatives of covariant velocity:

$$\zeta_{12}^{\text{ffc}} = D_1 u_2 - D_2 u_1, \tag{62}$$

$$\zeta_{13}^{\text{fcf}} = D_1 u_3 - D_3 u_1, \tag{63}$$

$$\zeta_{23}^{\text{cff}} = D_2 u_3 - D_3 u_2. \tag{64}$$

The metric does not appear in these formulas because covariant velocity is a one-form. Metric dependence should enter through Hodge maps, inner products, face areas, pressure/force projections, and transport diagnostics. This mirrors TRiSK and FEEC practice: incidence operators encode topology, while Hodge stars encode geometry [18, 21, 25, 24].

6.6 Numerical test campaign for staggered Hodge maps

The Hodge map should be selected empirically through a staged test campaign. This is not a substitute for theory; it is the way to ensure that the selected theory survives the actual Oceananigans staggered locations, halo fills, metric storage, and finite-volume operators.

Candidate scorecard. Each candidate Hodge map should be evaluated against the same scorecard:

Test	Target metric	Product interpolated	Energy symmetric
Hodge convergence			
Weighted adjointness			
Positive kinetic energy			
Free-stream preservation			
Orthogonal limit			
Linear-mode stability			
Centered energy conservation			
Tracer conservation			
WENO vorticity behavior			
Cost and memory			

The default should be the simplest Hodge map that passes free-stream preservation, positivity, adjointness, and orthogonal-limit tests. If neither simple candidate passes, use the energy-symmetric Hodge. If the energy-symmetric Hodge is too expensive, derive and test a mass-lumped approximation.

Metric and local consistency tests. For the equiangular gnomonic panel, test the analytic metric relations at every required location:

$$g_{ij}g^{jk} \approx \delta_i^k, \qquad g_{11}g_{22} - g_{12}^2 > 0, \qquad \sum_{\text{cells}} A_{\text{cell}} \approx \frac{4\pi R^2}{6}. \tag{65}$$

Then choose analytic physical vector fields, compute covariant components exactly, apply each Hodge map, and compare the resulting $\mathcal{V}^i$, u^i, and $u^i_\perp$ with exact values at $^{\text{fcc}}$, $^{\text{cfc}}$, and $^{\text{ccf}}$. Test constant Cartesian flow, solid-body rotation, and a smooth manufactured vector field.

Weighted adjointness. For each off-diagonal pair, compute

$$\epsilon_{\text{adj}} = \frac{\left\| W_1\mathcal{H}^{12} - (W_2\mathcal{H}^{21})^T \right\|}{\|W_1\mathcal{H}^{12}\| + \|(W_2\mathcal{H}^{21})^T\|}. \tag{66}$$

For a deliberately energy-symmetric Hodge, this should be near roundoff in Float64. For a mass-lumped Hodge, the defect should be small and should converge under refinement. If it does not, the Hodge has no clean kinetic-energy inner product.

Positive energy and null-space tests. Assemble the symmetric part of the kinetic-energy matrix:

$$\mathsf{E} = \frac{1}{2}\left(W\mathcal{H} + \mathcal{H}^T W\right). \tag{67}$$

The minimum eigenvalue should be positive on non-degenerate grids. Negative eigenvalues or grid-scale near-null modes indicate a likely computational-mode pathway.

Free-stream preservation. Choose a constant Cartesian velocity $\boldsymbol{u}_0$, compute $u_i = \boldsymbol{u}_0 \cdot \boldsymbol{a}_i$, form $\mathcal{V}^i = \mathcal{H}^i{}_j u_j$, and evaluate

$$\mathcal{D}_i\mathcal{V}^i. \tag{68}$$

This should vanish to roundoff on affine skew grids and converge rapidly on curved panels. Repeat with a constant scalar and verify

$$\mathcal{D}_i(\mathcal{V}^i c_0) \approx 0. \tag{69}$$

This is a red-line test: if a constant physical flow creates divergence, scalar advection will generate noise before nonlinear dynamics even starts.

Orthogonal limit. Construct a family of grids with $g^{12} = O(\varepsilon)$. As $\varepsilon \to 0$, the non-orthogonal Hodge map must reduce to the existing orthogonal formula:

$$\mathcal{V}^{1,\mathrm{fcc}} \to G^{11,\mathrm{fcc}} u_1^{\mathrm{fcc}}, \tag{70}$$

$$\mathcal{V}^{2,\mathrm{cfc}} \to G^{22,\mathrm{cfc}} u_2^{\mathrm{cfc}}. \tag{71}$$

At $\varepsilon = 0$, the new centered tendency should agree with the existing orthogonal Oceananigans operator to roundoff, modulo intentionally changed code paths.

Linear mode analysis. On a periodic affine skew grid, assemble the linearized shallow-water, acoustic, or incompressible projection operator and inspect eigenvalues and eigenvectors. With centered nondissipative operators, eigenvalues should be purely imaginary up to time-discretization effects, and there should be no growing checkerboard branches. Repeat on a smoothly varying skew grid to expose metric-placement errors. If a Hodge map passes affine tests but fails variable-metric tests, the issue is metric staggering rather than C-grid topology.

Centered conservation tests. Before WENO is enabled, test the centered non-orthogonal vector-invariant operator. With periodic boundaries and no forcing, the following should be conserved or have a controlled time-discretization residual:

global tracer mass, (72)

tracer variance for divergence-free transport, when using centered scalar advection, (73)

kinetic energy $E_h = \frac{1}{2} u^T W \mathcal{H} u$ for inviscid incompressible centered momentum, (74)

potential-enstrophy analogues for shallow-water reductions, when the required PV mass measure is available. (75)

Failures here should be attributed to metric/Hodge pairing before WENO is considered.

WENO-specific tests. Only after the centered operator passes the previous tests should WENO be activated. The WENO reconstruction should act on relative vorticity:

$$\widehat{Z}_{ij} = \mathcal{R}_{\mathrm{WENO}}[\zeta_{ij}] + \mathcal{R}_{\mathrm{smooth}}[f_{ij}], \tag{76}$$

not directly on $Z_{ij} = \zeta_{ij} + f_{ij}$. In small-Rossby-number flows, the smooth planetary vorticity can dominate the smoothness indicators if WENO sees Z directly. Reconstructing ζ ensures that nonlinear dissipation acts on the dynamically relevant relative-vorticity variance.

Hydrostatic split identity. For horizontal momentum, verify discretely that the hydrostatic WENO-style split is algebraically consistent with the full compressible transport identity:

$$D_3(\mathcal{U}^3 u_i) + u_i \mathcal{D}_{h,\mathcal{U}} = \mathcal{U}^3 D_3 u_i + u_i \mathcal{D}_{\mathcal{U}}. \tag{77}$$

This prevents double-counting divergence terms when comparing the full 3D vector-invariant form with the hydrostatic vertical-flux plus horizontal-divergence-flux form.

Suggested test files. A practical Oceananigans/Breeze test layout is

```
test/Grids/test_spherical_shell_grid_metrics.jl
test/Grids/test_equiangular_gnomonic_panel.jl

test/Operators/test_hodge_consistency.jl
test/Operators/test_hodge_adjointness.jl
test/Operators/test_hodge_positivity.jl
test/Operators/test_free_stream_preservation.jl
test/Operators/test_orthogonal_limit.jl
test/Operators/test_metric_identity.jl

test/Advection/test_nonorthogonal_scalar_transport.jl
test/Advection/test_nonorthogonal_vector_invariant_centered.jl
test/Advection/test_nonorthogonal_weno_vorticity_reconstruction.jl
test/Advection/test_hydrostatic_split_identity.jl

test/Models/test_breeze_nonorthogonal_compressible.jl
test/Models/test_hfsm_nonorthogonal_rigid_lid.jl
```

Model tests should not be used to choose the Hodge map until the operator tests are passing.

6.7 Decision tree for the Hodge stencil

The Hodge implementation should evolve by decision gates rather than by committing prematurely to a single interpolation stencil:

If analytic Hodge consistency fails: check metric placement, analytic mapping derivatives, and staggered indexing before tuning interpolation.

If consistency passes but free-stream preservation fails: test conservative metric identities and face-area closure. Product-interpolated or energy-symmetric Hodge maps may be preferable to target-metric multiplication.

If free-stream preservation passes but positivity fails: replace the pointwise Hodge with an energy-symmetric local mass-matrix Hodge, or symmetrize the off-diagonal blocks by weighted transpose.

If positivity passes but linear modes show checkerboards: inspect pressure-gradient/divergence adjointness and scalar-cell to face interpolation. The issue is likely an operator null space rather than WENO.

If centered conservation passes but WENO is noisy: keep the Hodge map fixed and change only reconstruction targets or smoothness indicators. In particular, reconstruct ζ, not $Z = \zeta + f$.

If single-panel tests pass but panel-boundary tests fail: treat panel connectivity as a vector/two-form remapping problem. Rotate or transform covariant components, transports, and vorticity forms consistently across halos; do not patch with scalar halo fills.

6.8 Code notation for this machinery

Struct names and persistent field names should be verbose. Mathematical notation should be reserved for local pointwise operator functions and derivation-facing helpers. For example, a materialized `NonOrthogonalVectorInvariant` should use fields such as

```
covariant_velocities
contravariant_velocities
transport_velocities
volume_transport
mass_transport
mass_transport_divergence
kinetic_energy
relative_vorticity
planetary_vorticity
absolute_vorticity
```

rather than persistent fields named `u_cov`, `U`, `C`, or `Z`. Internal formula functions may still use compact names such as $\zeta_{12}^{\mathrm{ffc}}$, Z_{12}^{ffc}, or $(\mathcal{D}_i\mathcal{U}^i)^{\mathrm{ccc}}$, because such names make the pointwise kernels read like the derivation.

7 Scalar, tracer, and mass transport

The scalar equation should be written in flux form:

$$\partial_t(J\rho\chi) + \mathcal{D}_i(\mathcal{U}^i\chi_i^{\uparrow}) = JS_\chi. \tag{78}$$

The velocity tuple passed to scalar advection should therefore be `transport_velocities(model)`, not necessarily `model.velocities`. This is already how HFSM is organized [8]. Breeze should use the same concept, either by adding a field `model.transport_velocities` or by formalizing the existing functional hook

```
transport_velocities(model) = model.velocities
```

with non-orthogonal and terrain-following specializations.

Two implementation paths are possible.

Velocity-based path. Diagnose $u_\perp^i$ and pass it as `transport_velocities`. Existing scalar kernels that multiply velocity by face area can be reused.

Flux-based path. Diagnose $\mathcal{U}^i$ or $\mathcal{V}^i$ and introduce scalar kernels that use transports directly. This is mathematically cleaner and avoids ambiguity about whether the transported object is raw contravariant velocity, area-normal velocity, volume transport, or mass transport.

The initial integration may use the velocity-based path in HFSM for minimal churn, while the long-term target should be a flux-based scalar advection API.

8 Three-dimensional vector-invariant momentum equation

The covariant momentum equation can be written in a compressible conservative vector-invariant form. Define the kinetic energy

$$K = \tfrac{1}{2}u_i u^i. \tag{79}$$

Define the relative vorticity two-form and the absolute-vorticity two-form

$$\zeta_{ij} = \partial_i u_j - \partial_j u_i, \qquad \zeta_{ij} = -\zeta_{ji}, \tag{80}$$

$$Z_{ij} = \zeta_{ij} + f_{ij}, \qquad Z_{ij} = -Z_{ji}. \tag{81}$$

Here f_{ij} is the planetary-vorticity two-form. The vector-invariant momentum equation uses the absolute-vorticity factor Z_{ij}. WENO reconstruction, however, should act on the relative vorticity ζ_{ij}, with f_{ij} added by a centered or otherwise smooth reconstruction. This prevents a smooth, large planetary-vorticity contribution from hiding dynamically important roughness in the relative vorticity at small Rossby number.

Using the identity

$$J\nabla_j(\rho u^j u_i) = J\rho\,\partial_i K - \mathcal{U}^j Z_{ij} + u_i \partial_j \mathcal{U}^j, \tag{82}$$

we obtain the advective tendency

$$\partial_t(J\rho u_i) = \sum_{j\neq i} \mathcal{U}^j Z_{ij} - J\rho\,\partial_i K - u_i \mathcal{D}_{\mathcal{U}} + \text{pressure, buoyancy, diffusion, forcing.} \tag{83}$$

The full three-dimensional divergence $\mathcal{D}_{\mathcal{U}} = \partial_j \mathcal{U}^j$ must include vertical flux divergence. This is the key difference from quasi-two-dimensional primitive-equation vector-invariant forms.

Compressible versus hydrostatic-incompressible regimes. In the fully compressible formulation, the conservative transport is the mass transport

$$\mathcal{U}^i = J\rho u^i, \tag{84}$$

and continuity is

$$\partial_t(J\rho) + \mathcal{D}_{\mathcal{U}} = 0, \qquad \mathcal{D}_{\mathcal{U}} = -\partial_t(J\rho). \tag{85}$$

Thus $\mathcal{D}_{\mathcal{U}}$ is generally nonzero. This is the fully compressible case targeted by Breeze.

For a hydrostatic-incompressible or Boussinesq implementation, the primary conservative transport is instead the compatible volume transport

$$\mathcal{V}^i = Ju^i, \tag{86}$$

and the fixed-coordinate incompressibility constraint is

$$\mathcal{D}_{\mathcal{V}} = \partial_i \mathcal{V}^i = 0. \tag{87}$$

If density is a constant ρ_0, then $\mathcal{U}^i = \rho_0 \mathcal{V}^i$ and $\mathcal{D}_{\mathcal{U}} = \rho_0 \mathcal{D}_{\mathcal{V}} = 0$. But the primitive object in HFSM-style incompressible dynamics is $\mathcal{V}^i$, not $\mathcal{U}^i$.

For moving coordinates such as a free-surface or z-star grid, the compatible statement is instead

$$\partial_t J + \partial_i \mathcal{V}^i = 0, \tag{88}$$

so a divergence-like correction may still appear in transformed coordinates even though the physical velocity field is incompressible. This is why the implementation should carry an explicit `transport_fluxes` object rather than relying on the name `velocities`.

Why the existing WENO vector-invariant vertical split is not an extra compressible divergence term. The current Oceananigans `WENOVectorInvariant` does not use the fully three-dimensional form (83) for hydrostatic horizontal momentum. Instead, for horizontal components $i = 1, 2$, it rewrites the vertical vorticity and vertical kinetic-energy-gradient contribution as a vertical momentum-flux divergence plus a horizontal divergence flux. This identity is exact for the conservative advective operator.

Let $h \in \{1, 2\}$, define

$$K_h = \tfrac{1}{2} u_h u^h, \qquad \mathcal{D}_{h,\mathcal{U}} = \partial_1 \mathcal{U}^1 + \partial_2 \mathcal{U}^2, \tag{89}$$

and write the positive conservative advective operator as

$$\mathcal{A}_i = J\nabla_j(\rho u^j u_i) = J\rho\,\partial_i K - \mathcal{U}^j Z_{ij} + u_i \mathcal{D}_{\mathcal{U}}. \tag{90}$$

For a vertically orthogonal thin shell, the vertical part of this identity satisfies

$$J\rho\,\partial_i K_3 - \mathcal{U}^3 Z_{i3} + u_i \mathcal{D}_{\mathcal{U}} = \mathcal{U}^3 \partial_3 u_i + u_i \mathcal{D}_{\mathcal{U}} \tag{91}$$

$$= \partial_3(\mathcal{U}^3 u_i) + u_i \mathcal{D}_{h,\mathcal{U}}, \tag{92}$$

where $K_3 = \frac{1}{2} u_3 u^3$. The equivalent compact identity is

$$\partial_3(\mathcal{U}^3 u_i) + u_i \mathcal{D}_{h,\mathcal{U}} = \mathcal{U}^3 \partial_3 u_i + u_i \mathcal{D}_{\mathcal{U}}. \tag{93}$$

This is the key cancellation: the $\mathcal{U}^3 \partial_i u_3$ part of the vertical vorticity flux cancels the horizontal gradient of vertical kinetic energy, while the remaining total mass divergence combines with vertical advection into a vertical momentum-flux divergence plus horizontal mass-divergence flux.

Therefore, for horizontal momentum,

$$\mathcal{A}_i = J\rho\,\partial_i K_h - \sum_{h=1}^{2} \mathcal{U}^h Z_{ih} + \partial_3(\mathcal{U}^3 u_i) + u_i \mathcal{D}_{h,\mathcal{U}}, \qquad i = 1, 2. \tag{94}$$

Equation (94) is the algebraic structure used by the existing WENO vector-invariant hydrostatic scheme: the vertical term is represented as a vertical momentum-flux divergence, while the horizontal divergence flux $u_i \mathcal{D}_{h,\mathcal{U}}$ is reconstructed or upwinded separately. In the incompressible limit, replace $\mathcal{U}^i$ by $\mathcal{V}^i$ and use $\mathcal{D}_{\mathcal{V}} = 0$. Then $\mathcal{D}_{h,\mathcal{V}} = -\partial_3 \mathcal{V}^3$, and

$$\partial_3(\mathcal{V}^3 u_i) + u_i \mathcal{D}_{h,\mathcal{V}} = \mathcal{V}^3 \partial_3 u_i, \tag{95}$$

the ordinary vertical advection of horizontal momentum.

For compressible dynamics, $\mathcal{D}_{\mathcal{U}} \neq 0$, but (94) remains an exact conservative split if $\mathcal{U}^i = J\rho u^i$. The important rule is not to add both formulations. One must choose either the full three-dimensional form (83), which contains $-u_i \mathcal{D}_{\mathcal{U}}$ in the tendency, or the hydrostatic horizontal split (94), which contains $-\partial_3(\mathcal{U}^3 u_i) - u_i \mathcal{D}_{h,\mathcal{U}}$ in the tendency. Adding a separate full $-u_i \mathcal{D}_{\mathcal{U}}$ term on top of the vertical-flux/horizontal-divergence split would double count part of the conservative product rule.

The three-dimensional two-form set is

$$Z_{12}, \qquad Z_{13}, \qquad Z_{23}. \tag{96}$$

Thus vertical momentum advection is not a separate flux-form special case in the full three-dimensional form. It enters through Z_{13}, Z_{23}, and the transports $\mathcal{U}^1$,$\mathcal{U}^2$,$\mathcal{U}^3$. For example, the first covariant momentum tendency includes

$$\mathcal{U}^2 Z_{12} + \mathcal{U}^3 Z_{13}, \tag{97}$$

and the third covariant momentum tendency includes

$$\mathcal{U}^1 Z_{31} + \mathcal{U}^2 Z_{32}. \tag{98}$$

9 Zuasi-two-dimensional and hydrostatic reductions

A quasi-two-dimensional or hydrostatic form should be a reduction of the three-dimensional formula, not a separate derivation. If the model prognoses only horizontal momentum, then $i = 1, 2$. There are two algebraically equivalent ways to write the horizontal advective tendency.

The first is the direct full-vector-invariant reduction:

$$\partial_t(J\rho u_1) = \mathcal{U}^2 Z_{12} + \mathcal{U}^3 Z_{13} - J\rho\,\partial_1 K - u_1 \mathcal{D}_{\mathcal{U}} + \cdots, \tag{99}$$

$$\partial_t(J\rho u_2) = \mathcal{U}^1 Z_{21} + \mathcal{U}^3 Z_{23} - J\rho\,\partial_2 K - u_2 \mathcal{D}_{\mathcal{U}} + \cdots. \tag{100}$$

This form is natural for a fully compressible three-dimensional vector-invariant implementation, especially when vertical momentum is also prognosed and K_3, Z_{13}, and Z_{23} are available as explicit fields.

The second is the hydrostatic horizontal split obtained from (94):

$$\partial_t(J\rho u_i) = \sum_{h=1}^{2} \mathcal{U}^h Z_{ih} - J\rho\,\partial_i K_h - \partial_3(\mathcal{U}^3 u_i) - u_i \mathcal{D}_{h,\mathcal{U}} + \cdots, \qquad i = 1, 2. \tag{101}$$

This form is closer to the current Oceananigans `WENOVectorInvariant`: the vertical transport of horizontal momentum is treated as a vertical momentum-flux divergence, while the horizontal divergence flux is reconstructed and upwinded. For fixed-coordinate hydrostatic-incompressible flow, use volume transports $\mathcal{V}^i$. The total volume-transport divergence $\mathcal{D}_{\mathcal{V}}$ vanishes, but the horizontal divergence $\mathcal{D}_{h,\mathcal{V}}$ is not zero in general; it is the negative vertical volume-flux divergence. Therefore the horizontal divergence flux in (101) is still needed even in an incompressible model.

For HFSM, the immediate target should be (101) with volume transports $\mathcal{V}^i$, tracer advection through `model.transport_velocities`, and no separate full-divergence correction because $\mathcal{D}_{\mathcal{V}} = 0$ in the fixed-coordinate physical incompressible constraint. For compressible Breeze dynamics, either (100) or (101) can be used, but they must not be mixed. A fully three-dimensional compressible momentum system should prefer (100); a hydrostatic compressible horizontal-momentum system can use (101) with mass transports $\mathcal{U}^i$.

10 Algorithmic abstraction

The existing Oceananigans pattern is

```
VectorInvariant(...)
WENOVectorInvariant(...)
```

where `WENOVectorInvariant` is a convenience constructor that fills the generic vector-invariant object with WENO sub-schemes. The current object stores reconstruction configuration, not runtime diagnostic fields [9, 12, 13].

The non-orthogonal extension should follow the same naming pattern:

```
NonOrthogonalVectorInvariant(...)
NonOrthogonalWENOVectorInvariant(...)
```

The first is the reconstruction-agnostic geometric operator; the second constructs the first with WENO reconstruction schemes. A centered, energy-conserving, or lower-order scheme can then be tested with the same geometric operator:

```
NonOrthogonalVectorInvariant(
    vorticity_scheme = Centered(order=2),
    shear_vorticity_scheme = Centered(order=2),
    divergence_scheme = Centered(order=2),
    kinetic_energy_gradient_scheme = Centered(order=2),
)

NonOrthogonalWENOVectorInvariant(
    vorticity_order = 9,
    shear_vorticity_order = 5,
    divergence_order = 5,
    kinetic_energy_gradient_order = 5,
)
```

Unlike the existing `WENOVectorInvariant`, the materialized non-orthogonal object should own geometric diagnostic fields at top level:

```
struct NonOrthogonalVectorInvariant{...} <: AbstractAdvectionScheme{N, FT}
    # reconstruction choices
    vorticity_scheme
    shear_vorticity_scheme
    divergence_scheme
    kinetic_energy_gradient_scheme
    transport_scheme
    upwinding

    # materialized geometric state
    u_cov                # u_i
    u_con                # u^i
    transport_velocities # u_perp^i if velocity-based scalar kernels are used
    volume_transports    # calV^i = J u^i
    mass_transports      # calU^i = J rho u^i
    div_volume_transport # D_i calV^i
    div_mass_transport   # D_i calU^i
    kinetic_energy       # K = 1/2 u_i u^i
    vorticity_two_forms  # Z12, Z13, Z23
end
```

These are not incidental scratch arrays. They are the metric-compatible state variables shared by scalar advection, continuity, and vector-invariant momentum advection.

11 Materialization and update lifecycle

Because the top-level diagnostic fields depend on the grid, architecture, precision, halo sizes, and field locations, allocation should happen during advection materialization:

```
function materialize_advection(scheme::NonOrthogonalVectorInvariant, grid)
    return NonOrthogonalVectorInvariant(
        materialize_advection(scheme.vorticity_scheme, grid),
        materialize_advection(scheme.shear_vorticity_scheme, grid),
        materialize_advection(scheme.divergence_scheme, grid),
        materialize_advection(scheme.kinetic_energy_gradient_scheme, grid),
        materialize_advection(scheme.transport_scheme, grid),
        materialize_advection(scheme.upwinding, grid),
```

```
        allocate_nonorthogonal_fields(grid)...,
    )
end
```

This extends Oceananigans' current `materialize_advection` mechanism, which already recurses into sub-schemes and specializes WENO backend choices [13].

A typical tendency-evaluation lifecycle is

```
fill_halo_regions!(model.momentum)
fill_halo_regions!(density)

update_nonorthogonal_vector_invariant_fields!(
    model.advection.momentum,
    model.grid,
    density,
    model.momentum,
    model.velocities,
)

# Momentum uses model.velocities and the non-orthogonal VI fields.
compute_momentum_tendencies!(model)

# Scalars use transport_velocities(model) or transport_fluxes(model).
compute_scalar_tendencies!(model)
```

Inside the update:

```
compute_covariant_velocities!(advection.u_cov, model.momentum, density, grid)
compute_contravariant_velocities!(advection.u_con, advection.u_cov, grid)
compute_transport_velocities!(advection.transport_velocities, advection.u_con, grid)
compute_transport_fluxes!(advection.volume_transports, advection.mass_transports,
    density, advection.u_con, grid)
compute_divergences!(advection.div_volume_transport, advection.div_mass_transport,
    advection.volume_transports, advection.mass_transports, grid)
compute_kinetic_energy!(advection.kinetic_energy, advection.u_cov, advection.u_con, grid)

compute_vorticity_two_forms!(advection.vorticity_two_forms, advection.u_cov, grid)
```

For orthogonal grids, `transport_velocities(model)` can alias `model.velocities`. For non-orthogonal grids, it should return the transport-velocity tuple owned by the materialized momentum advection object. Likewise, `transport_fluxes(model)` should return the same object's mass transports in compressible models and volume transports in Boussinesq or incompressible models.

12 HydrostaticFreeSurfaceModel integration

HFSM already has the important separation:

Momentum tendencies use `model.velocities`.

Tracer tendencies use `model.transport_velocities`.

The non-orthogonal HFSM integration should therefore focus on constructing and updating the correct `model.transport_velocities`, or making `model.transport_velocities` a reference to the materialized non-orthogonal advection object's transport velocity fields. A separate

`transport_fluxes(model)` or `volume_transports(model)` accessor may be useful for continuity, free-surface, and momentum coupling.

A possible HFSM convention is

```
model.velocities           # velocity fields for momentum dynamics and diagnostics
model.transport_velocities # scalar-advection transport velocities
model.advection.momentum   # NonOrthogonalVectorInvariant owning geometry fields
```

For orthogonal grids, `model.transport_velocities === model.velocities` is allowed. For non-orthogonal grids, `model.transport_velocities` should be updated from the materialized momentum-advection transport fields before tracer tendencies are computed.

HFSM should start with a Boussinesq or hydrostatic volume-transport version of the hydrostatic split (101). The compressible mass-transport version is needed for Breeze. The distinction is whether the conservative transport is $\mathcal{V}^i = Ju^i$ or $\mathcal{U}^i = J\rho u^i$, and whether the relevant divergence is $\mathcal{D}_\mathcal{V}$ or $\mathcal{D}_\mathcal{U}$.

13 Breeze.AtmosphereModel integration

Breeze should adopt the HFSM split through functions:

```
transport_velocities(model) = model.velocities
transport_fluxes(model) = advecting_momentum(model)
```

with non-orthogonal methods

```
transport_velocities(model::NonOrthogonalAtmosphereModel) =
    model.advection.momentum.transport_velocities

transport_fluxes(model::NonOrthogonalAtmosphereModel) =
    model.advection.momentum.mass_transports
```

The existing terrain-following compressible implementation already diagnoses contravariant vertical velocity and momentum and routes scalar transport and density divergence through those diagnostics [2]. The non-orthogonal extension generalizes that from one vertical component to all three components.

For compressible Breeze dynamics, the density equation should use the same transport fluxes as scalar advection:

$$\partial_t(J\rho) = -\mathcal{D}_i\mathcal{U}^i = -\mathcal{D}_\mathcal{U}. \tag{102}$$

The current compressible density tendency uses the negative divergence of momentum on standard grids [3]. On a non-orthogonal grid, it must dispatch to `transport_fluxes(model)` or `mass_transports(model)` rather than to covariant momentum.

14 Grid data structure

Use the single public grid name `SphericalShellGrid`. Orthogonal and non-orthogonal behavior should be a property of the mapping and metric provider, not two unrelated public grid families. Oceananigans already has abstract curvilinear and horizontally curvilinear grid types [15, 16]. The concrete non-orthogonal spherical-shell grid can fit into that hierarchy.

A minimal structure is

```
struct SphericalShellGrid{FT, TX, TY, TZ, Z, Map, Metrics, Arch} <:
       AbstractHorizontallyCurvilinearGrid{FT, TX, TY, TZ, Z, Arch}
    architecture :: Arch
    Nx :: Int; Ny :: Int; Nz :: Int
    Hx :: Int; Hy :: Int; Hz :: Int
    Lz :: FT
    z :: Z
    radius :: FT
    mapping :: Map
    metrics :: Metrics
end
```

The metric object should expose at least

```
struct NonOrthogonalSphericalShellMetrics{Coords, Areas, Volumes, GCov, GCon}
    coordinates :: Coords
    face_areas  :: Areas
    volumes     :: Volumes
    gcov        :: GCov
    gcon        :: GCon
end
```

For the first equiangular panel, store center metrics and the face metrics required by Hodge maps. Avoid prematurely storing every staggered metric combination until tests demonstrate they are needed.

The geometry tests should precede dynamics. They include:

$$g_{\alpha\beta} \neq 0 \quad \text{away from panel symmetry axes,} \tag{103}$$

$$g_{\alpha\beta} = 0 \quad \text{on panel symmetry axes,} \tag{104}$$

$$g_{\alpha\alpha} g_{\beta\beta} - g_{\alpha\beta}^2 > 0, \tag{105}$$

$$\sum_{\text{one panel cells}} A \approx \frac{4\pi R^2}{6}, \tag{106}$$

$$\sum_{\text{six panel cells}} A \approx 4\pi R^2. \tag{107}$$

A finite-volume metric-identity test should verify that oriented face-area vectors close around each cell, so that constant transport does not generate spurious divergence.

15 Relationship to existing WENO vector-invariant advection

The existing `WENOVectorInvariant` is best viewed as a stateless reconstruction configuration. It stores WENO sub-schemes, smoothness stencils, and upwinding choices; it does not store vorticity fields, divergence fields, kinetic energy fields, or WENO-weight scratch arrays [9, 12, 10]. Zuantities such as ζ_3, divergence smoothness, and kinetic-energy-gradient terms are evaluated pointwise inside tendency kernels.

For hydrostatic horizontal momentum, the present WENO vector-invariant scheme uses the split in (101). In code, the upwinded vertical contribution is assembled as a vertical momentum-flux divergence plus an upwinded horizontal divergence flux: `vertical_advection_U` and `vertical_advection_V` add `dz(... _advective_momentum_flux_Wu/Wv ...)` to `upwinded_divergence_flux_U/V`. The

latter uses horizontal flux differences such as `delta_x_U` and `delta_y_V`; the smoothness indicators can be based on horizontal divergence, while the momentum inside the vertical flux is reconstructed by the vertical advection scheme [9, 11]. Thus the phrase "divergence upwinding" in the existing scheme refers to upwinding the horizontal divergence flux $u_i \mathcal{D}_{h,\mathcal{V}}$ or $u_i \mathcal{D}_{h,\mathcal{U}}$, depending on the transport used. It does not mean adding a nonzero total divergence to an incompressible model.

This distinction matters for the compressible extension. If the new non-orthogonal scheme is written in full three-dimensional vector-invariant form, the tendency contains $-u_i \mathcal{D}_{\mathcal{U}}$. If instead the scheme follows the hydrostatic WENO split, the tendency contains $-\partial_3(\mathcal{U}^3 u_i) - u_i \mathcal{D}_{h,\mathcal{U}}$. These are equivalent only when the vertical vorticity and vertical kinetic-energy pieces are handled consistently, as in (92). The implementation should therefore expose this as a design choice:

Full 3D compressible VI use Z_{12}, Z_{13}, Z_{23}, full K, and full mass-transport divergence $\mathcal{D}_{\mathcal{U}}$.

Hydrostatic horizontal split use horizontal Z_{12}, horizontal kinetic energy K_h, vertical momentum-flux divergence, and horizontal divergence flux $\mathcal{D}_{h,\mathcal{U}}$ or $\mathcal{D}_{h,\mathcal{V}}$, with no additional full divergence term.

The non-orthogonal scheme should reuse the existing reconstruction logic where possible, but the geometric state is more important and more expensive. The Hodge map $u_i \mapsto u^i$, the volume or mass transport $\mathcal{V}^i$ or $\mathcal{U}^i$, and the kinetic energy $K = \frac{1}{2} u_i u^i$ must be shared by mass, tracer, energy, and momentum tendencies. This justifies the materialized top-level diagnostic fields in `NonOrthogonalVectorInvariant`.

16 Test hierarchy

The recommended test order is:

1. **Analytic geometry tests.** Compare numerical metric fields on the equiangular panel to (15)–(20).
2. **Metric identity tests.** Verify face-area closure and zero divergence for constant compatible fluxes.
3. **Orthogonal limit tests.** On rectilinear or orthogonal spherical-shell grids, the non-orthogonal operator should agree with the existing orthogonal operator to truncation error.
4. **Passive scalar tests.** Advect smooth and nonsmooth tracers using `transport_velocities` and verify conservation and convergence.
5. **Centered vector-invariant tests.** Test `NonOrthogonalVectorInvariant` with centered reconstruction before WENO.
6. **WENO vector-invariant tests.** Enable `NonOrthogonalWENOVectorInvariant` and compare stability, convergence, and dissipation.
7. **Model tests.** Run decaying two-dimensional turbulence, forced three-dimensional turbulence, and baroclinic adjustment on orthogonal and non-orthogonal grids.
8. **Six-panel tests.** Add panel connectivity only after single-panel geometry and transport are correct.

17 Principal risks and theory-facing mitigations

Ambiguity among velocity, area-normal velocity, volume flux, and mass flux. The mitigation is to require every scalar advection path to state whether it consumes $u^i_\perp$, $\mathcal{V}^i$, or $\mathcal{U}^i$. The decision tree is: if existing velocity-based kernels are reused, compute $u^i_\perp$; if new non-orthogonal kernels are introduced, consume $\mathcal{U}^i$ or $\mathcal{V}^i$ directly.

Incorrect Hodge maps at staggered locations. Cross-component interpolation in equations such as (33) is part of the metric operator. It should be tested independently by mapping analytic vector fields from physical space to covariant components and back to transport fluxes.

Edge locations for Z_{12}, Z_{13}, Z_{23}. Oceananigans does not currently expose all edge-field types as prominently as face and center fields. The first implementation should use explicit `Field\{Face,Face,Center\}`, `Field\{Face,Center,Face\}`, and `Field\{Center,Face,Face\}` objects if available, or add a small helper layer for two-form storage.

WENO smoothness indicators on non-orthogonal grids. The existing Silvestri-style WENO vector-invariant scheme separates reconstruction targets from smoothness stencils [20]. The non-orthogonal scheme should initially use conservative centered reconstruction to validate geometry, then introduce WENO with smoothness indicators based on covariant velocity, transport flux divergence, or physically scaled two-forms.

Single-panel boundary effects. One-panel turbulence tests with bounded sides can be contaminated by boundaries. Use periodic planar skewed grids for early turbulence tests, then single-panel scalar tests, then six-panel spherical tests.

HFSM/Breeze divergence. HFSM has a field-level `model.transport_velocities`; Breeze currently has a function-level `transport_velocities(model)`. The theory does not require one representation. The implementation should preserve the split and choose whichever representation minimizes churn in each model.

18 Summary

The non-orthogonal C-grid program should be built around a clean separation of objects:

$$u_i \quad \text{covariant velocity for momentum and vorticity,} \tag{108}$$

$$u^i \quad \text{contravariant velocity from the Hodge map,} \tag{109}$$

$$u^i_\perp \quad \text{area-normal transport velocity for velocity-based scalar kernels,} \tag{110}$$

$$\mathcal{U}^i = J\rho u^i \quad \text{conservative mass transport for continuity and compressible coupling.} \tag{111}$$

The theory supports a generic `NonOrthogonalVectorInvariant` operator and a WENO convenience constructor `NonOrthogonalWENOVectorInvariant`. The grid program should start with one equiangular gnomonic cubed-sphere panel, using analytic metrics and metric-identity tests before model dynamics. The code program should reuse the existing HFSM distinction between `velocities` and `transport_velocities`, and should formalize the same distinction in Breeze. Once the geometry and transport objects are correct, the full three-dimensional vector-invariant form (83) provides a unified path for horizontal and vertical momentum advection on non-orthogonal C-grids.

References

[1] NumericalEarth (2026a). *Breeze.jl dynamics interface*. https://github.com/NumericalEarth/Breeze.jl/blob/main/src/AtmosphereModels/dynamics_interface.jl.

[2] NumericalEarth (2026b). *Breeze.jl terrain-following compressible physics*. https://github.com/NumericalEarth/Breeze.jl/blob/main/src/CompressibleEquations/terrain_compressible_physics.jl.

[3] NumericalEarth (2026c). *Breeze.jl compressible density tendency*. https://github.com/NumericalEarth/Breeze.jl/blob/main/src/CompressibleEquations/compressible_density_tendency.jl.

[4] Climate Modeling Alliance (2026a). *ClimaAtmos.jl simulation grid constructors*. https://github.com/CliMA/ClimaAtmos.jl/blob/main/src/simulation/grids.jl.

[5] Climate Modeling Alliance (2026b). *ClimaCore.jl cubed-sphere meshes*. https://github.com/CliMA/ClimaCore.jl/blob/main/src/Meshes/cubedsphere.jl.

[6] Climate Modeling Alliance (2026c). *ClimaCore.jl common grid constructors*. https://github.com/CliMA/ClimaCore.jl/blob/main/src/CommonGrids/CommonGrids.jl.

[7] Lin, S.-J. and Rood, R. B. (1996). Multidimensional flux-form semi-Lagrangian transport schemes. *Monthly Weather Review*, 124(9), 2046–2070.

[8] Climate Modeling Alliance (2026d). *Oceananigans.jl HydrostaticFreeSurfaceModel tendency code*. https://github.com/CliMA/Oceananigans.jl/blob/main/src/Models/HydrostaticFreeSurfaceModels/compute_hydrostatic_free_surface_tendencies.jl.

[9] Climate Modeling Alliance (2026e). *Oceananigans.jl vector-invariant advection implementation*. https://github.com/CliMA/Oceananigans.jl/blob/main/src/Advection/vector_invariant_advection.jl.

[10] Climate Modeling Alliance (2026f). *Oceananigans.jl vector-invariant upwinding implementation*. https://github.com/CliMA/Oceananigans.jl/blob/main/src/Advection/vector_invariant_upwinding.jl.

[11] Oceananigans.jl developers. *Oceananigans.jl vector-invariant self-upwinding implementation*. https://github.com/CliMA/Oceananigans.jl/blob/main/src/Advection/vector_invariant_self_upwinding.jl.

[12] Climate Modeling Alliance (2026g). *Oceananigans.jl WENO reconstruction implementation*. https://github.com/CliMA/Oceananigans.jl/blob/main/src/Advection/weno_reconstruction.jl.

[13] Climate Modeling Alliance (2026h). *Oceananigans.jl advection materialization*. https://github.com/CliMA/Oceananigans.jl/blob/main/src/Advection/materialize_advection.jl.

[14] Climate Modeling Alliance (2026i). *Oceananigans.jl conformal cubed-sphere panel grid*. https://github.com/CliMA/Oceananigans.jl/blob/main/src/OrthogonalSphericalShellGrids/conformal_cubed_sphere_panel.jl.

[15] Climate Modeling Alliance (2026j). *Oceananigans.jl grid module*. https://github.com/CliMA/Oceananigans.jl/blob/main/src/Grids/Grids.jl.

[16] Climate Modeling Alliance (2026k). *Oceananigans.jl abstract grid definitions*. `https://github.com/CliMA/Oceananigans.jl/blob/main/src/Grids/abstract_grid.jl`.

[17] Rančić, M., Purser, R. J., and Mesinger, F. (1996). A global shallow-water model using an expanded spherical cube: gnomonic versus conformal coordinates. *Quarterly Journal of the Royal Meteorological Society*, 122(532), 959–982.

[18] Ringler, T. D., Thuburn, J., Klemp, J. B., and Skamarock, W. C. (2010). A unified approach to energy conservation and potential vorticity dynamics for arbitrarily-structured C-grids. *Journal of Computational Physics*, 229(9), 3065–3090.

[19] Ronchi, C., Iacono, R., and Paolucci, P. S. (1996). The "Cubed Sphere": a new method for the solution of partial differential equations in spherical geometry. *Journal of Computational Physics*, 124(1), 93–114.

[20] Silvestri, S., Wagner, G. L., Campin, J.-M., Constantinou, N. C., Hill, C. N., Souza, A. N., and Ferrari, R. (2024). A new WENO-based momentum advection scheme for simulations of ocean mesoscale turbulence. *Journal of Advances in Modeling Earth Systems*, 16(7), e2023MS004130.

[21] Thuburn, J. and Cotter, C. J. (2012). A framework for mimetic discretization of the rotating shallow-water equations on arbitrary polygonal grids. *SIAM Journal on Scientific Computing*, 34(3), B203–B225.

[22] Arakawa, A. and Lamb, V. R. (1981). A potential enstrophy and energy conserving scheme for the shallow water equations. *Monthly Weather Review*, 109(1), 18–36.

[23] Eldred, C. and Randall, D. (2016). Total energy and potential enstrophy conserving schemes for the shallow water equations using Hamiltonian methods: Derivation and properties. *arXiv:1609.03797*.

[24] McRae, A. T. T. and Cotter, C. J. (2013). Energy- and enstrophy-conserving schemes for the shallow-water equations, based on mimetic finite elements. *arXiv:1305.4477*.

[25] Thuburn, J. and Cotter, C. J. (2014). A primal-dual mimetic finite element scheme for the rotating shallow water equations on polygonal spherical meshes. *arXiv:1409.8130*.

[26] Climate Modeling Alliance (2026l). *ClimaCore.jl spectral-element grid and local geometry construction*. `https://github.com/CliMA/ClimaCore.jl/blob/main/src/Grids/spectralelement.jl`.